SE- Lab 7

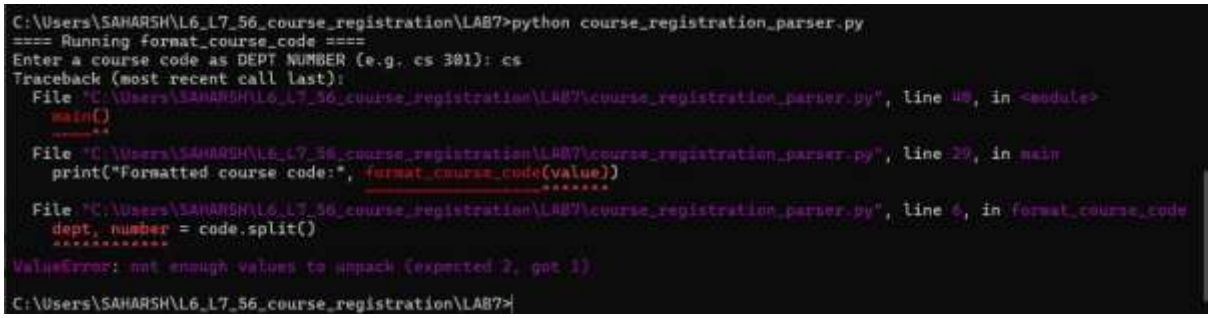
Name: Yerramilli Siva Sai Saharsh

SRN: PES2UG24CS618

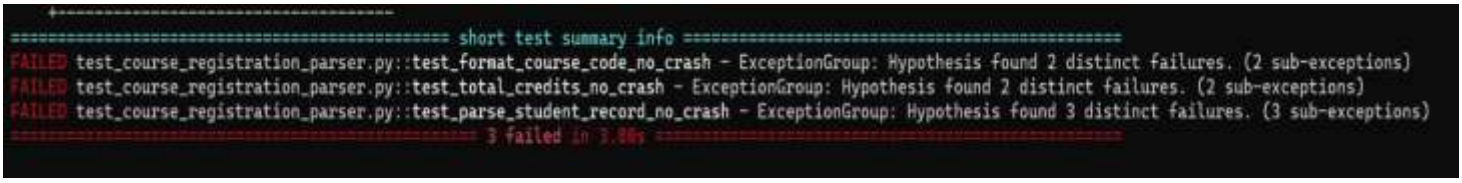
Section: J

Fuzzing for Input Validation Bugs

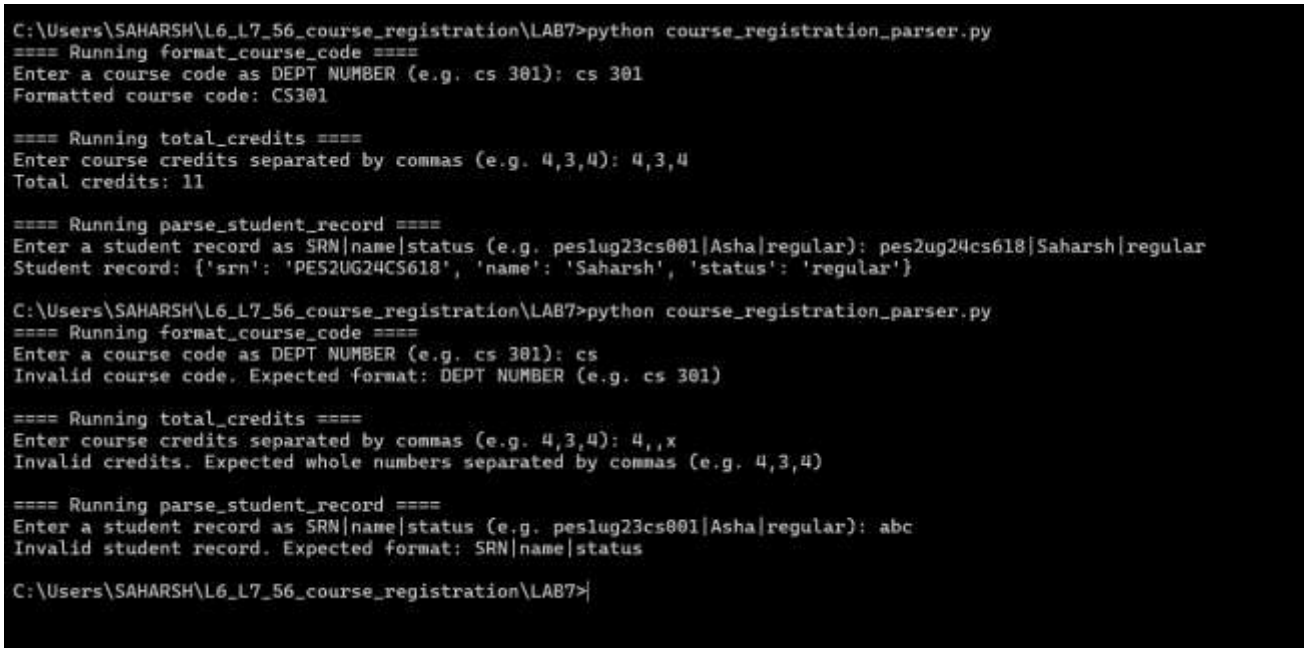
1. Buggy program crashing on invalid input



2. Hypothesis tests failing on buggy code (3 failed)



3. Fixed program handling valid and invalid input



4. Hypothesis tests passing on fixed code (3 passed)



5. Bugs found and fixes

Function	Bug found by fuzzing	Fix
format_course_code	None → AttributeError; '', 'cs' or 3+ words → ValueError (unpacking)	Check it's a string with exactly 2 words, letters-only department, digits-only number; else return None
total_credits	None → AttributeError; '', '4,,3', 'a' → ValueError from int()	Check type, strip each value, accept only ASCII digits, catch ValueError; else return None

parse_student_record	None → AttributeError; '' or `'a	b'→IndexError` (missing fields)

6. Reflection Questions

1. What bugs did fuzz testing reveal in the course registration parser, and why were they missed by the original code?

Hypothesis found crashes in all three functions. format_course_code crashed with ValueError on inputs like '' or 'cs' because it assumed the input always splits into exactly two words, and with an AttributeError on None. total_credits crashed with a ValueError on '' or '4,,3' because int() cannot convert empty or non-numeric strings. parse_student_record crashed with an IndexError on '' or '|' because it accessed parts[1] and parts[2] without checking that three fields existed. All three functions shared one root cause: they assumed every input would be well formed, as their docstrings stated, and never validated it. Testing only with the example inputs would never reveal these failures.

2. How is property-based fuzzing with Hypothesis different from traditional unit testing, and what are its advantages?

In a traditional unit test, I choose specific inputs and expected outputs, such as checking that format_course_code("cs 301") returns "CS301". This only tests the cases I think of. With Hypothesis, I describe a property that should hold for all inputs (here, "the function should never crash") and Hypothesis generates hundreds of random inputs, including empty strings, unusual Unicode characters, and None. When a test fails, Hypothesis shrinks the failing input to the simplest example, such as '' or '|', which makes the bug easy to understand. The main advantage is that it finds edge cases I would not have thought to write tests for. Its limitation is that a "no crash" property does not check whether the output is correct, so it works best alongside regular unit tests.

3. What defensive programming techniques did you use to fix the bugs, and how do they improve the robustness and security of the code?

I added input validation at the start of each function: checking that the input is a string, that it has the expected number of parts (two words for a course code, three fields for a student record), and that values have the right format (letters only for the department, digits only for course numbers and credits). Instead of raising an exception, each function returns None for invalid input, and main() shows a clear error message rather than a traceback. I also wrapped the integer conversion in a try/except ValueError to handle extreme cases like very long numbers. These changes mean the program fails gracefully instead of crashing, which improves the user experience.

They also improve security, because unhandled crashes can enable denial-of-service attacks and tracebacks can leak internal details of the program to an attacker.